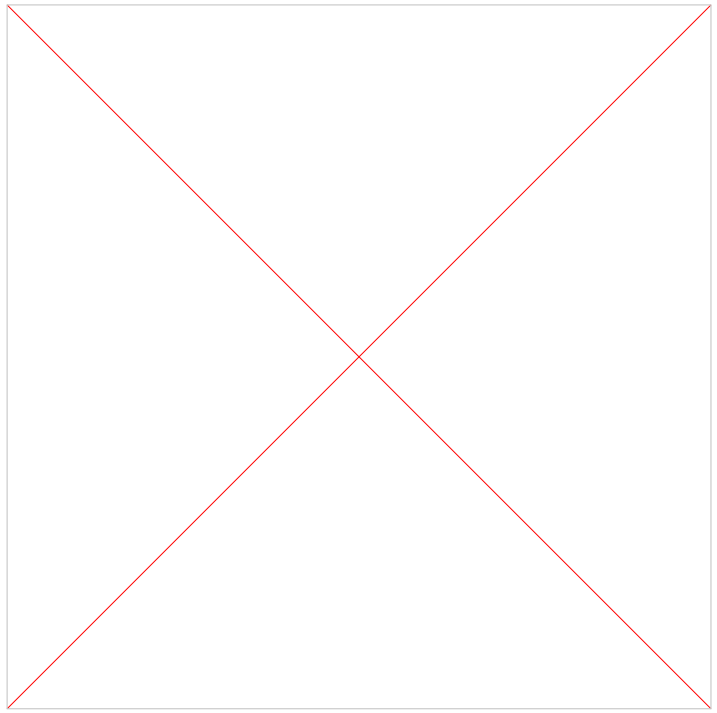


JavaScript

Duangpen Jetpipattanapong



introduction

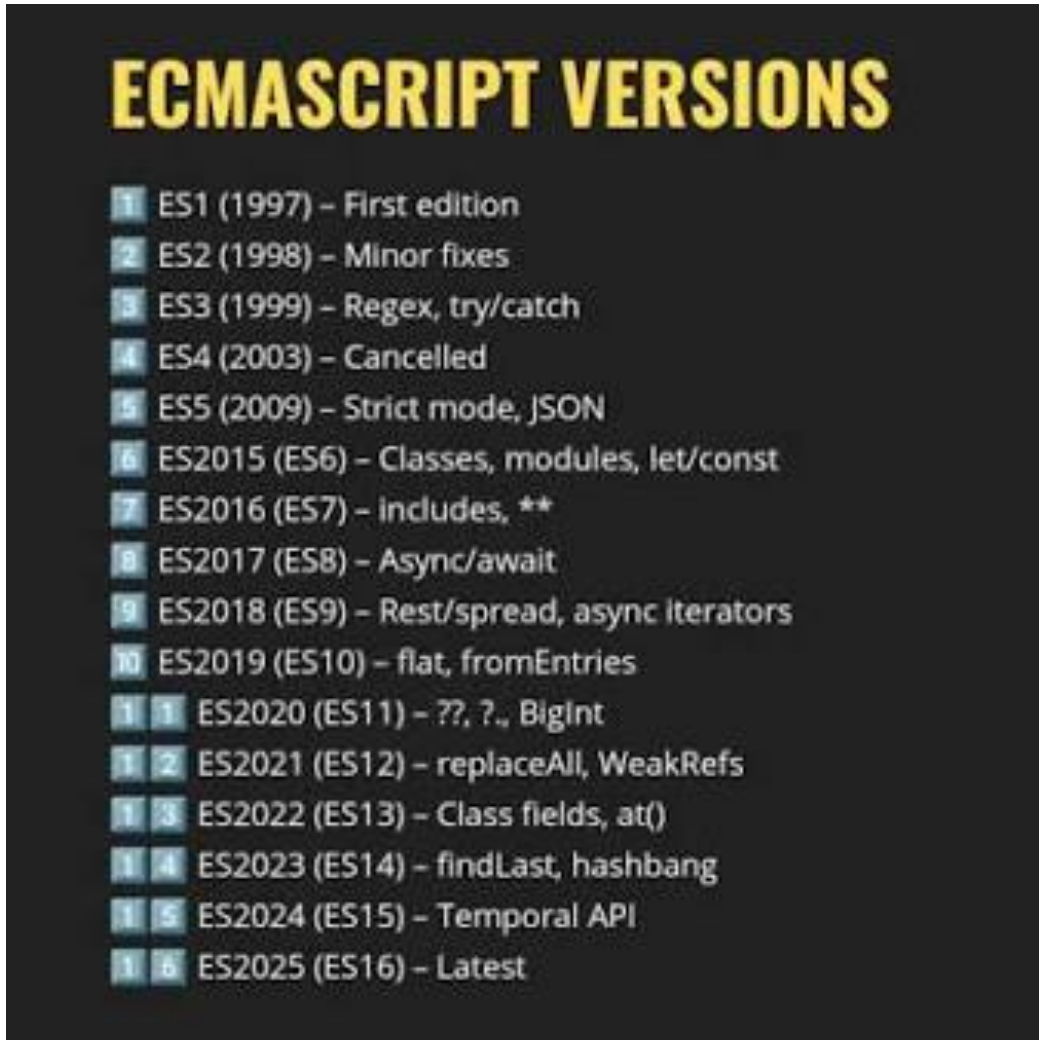
JavaScript

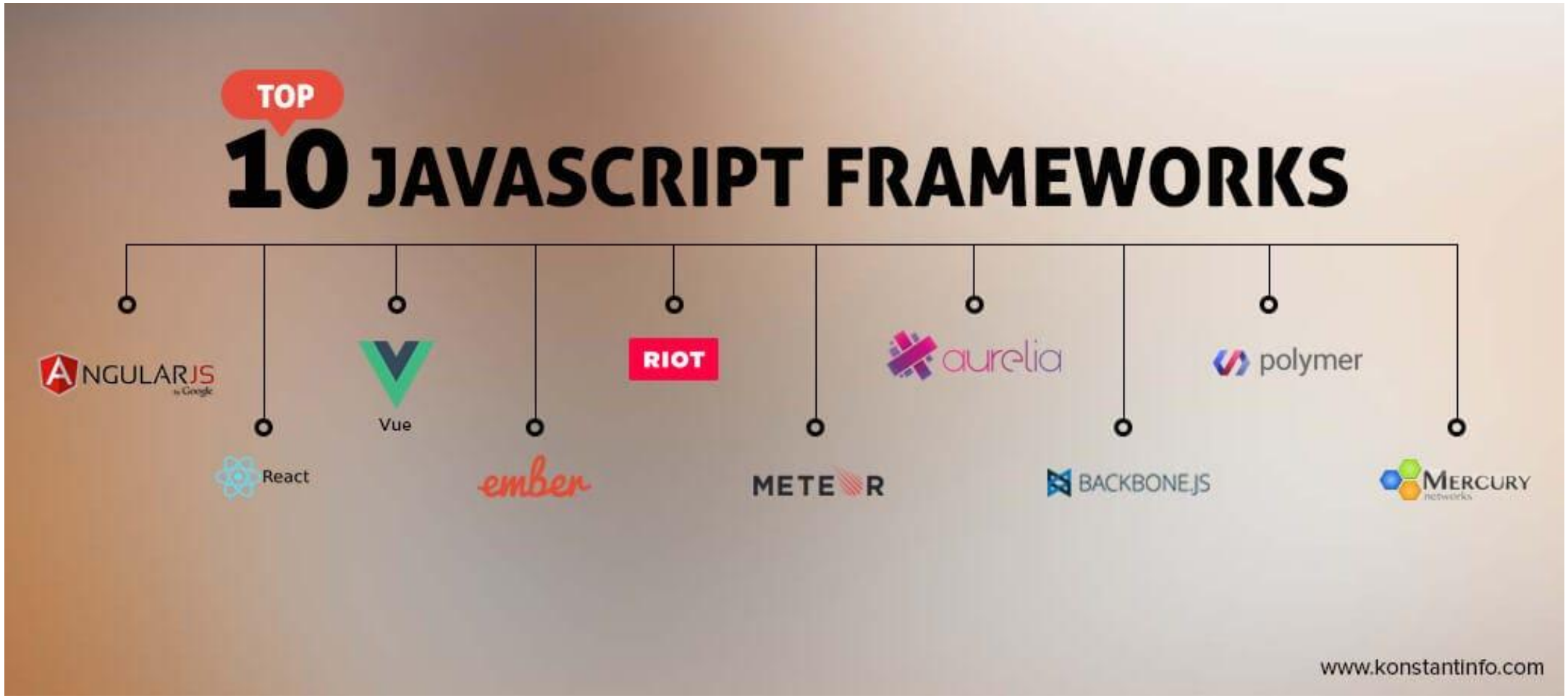
JavaScript is the programming language of the web.

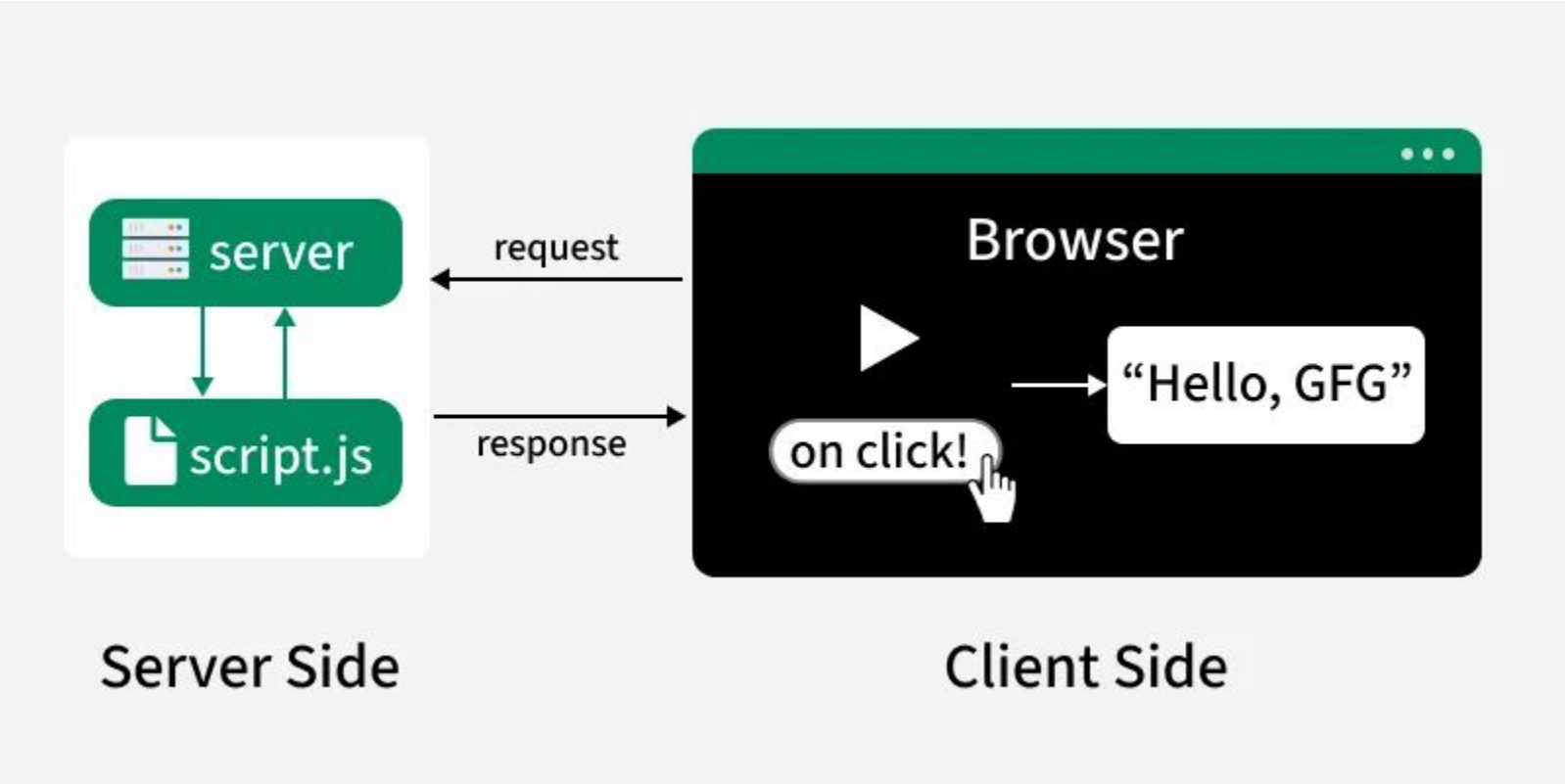
ECMAScript is the official name of the **JavaScript standard**

From 1997 versions was abbreviated by numbers. (**ES1, ES2, ES3, ES5, ES6**)

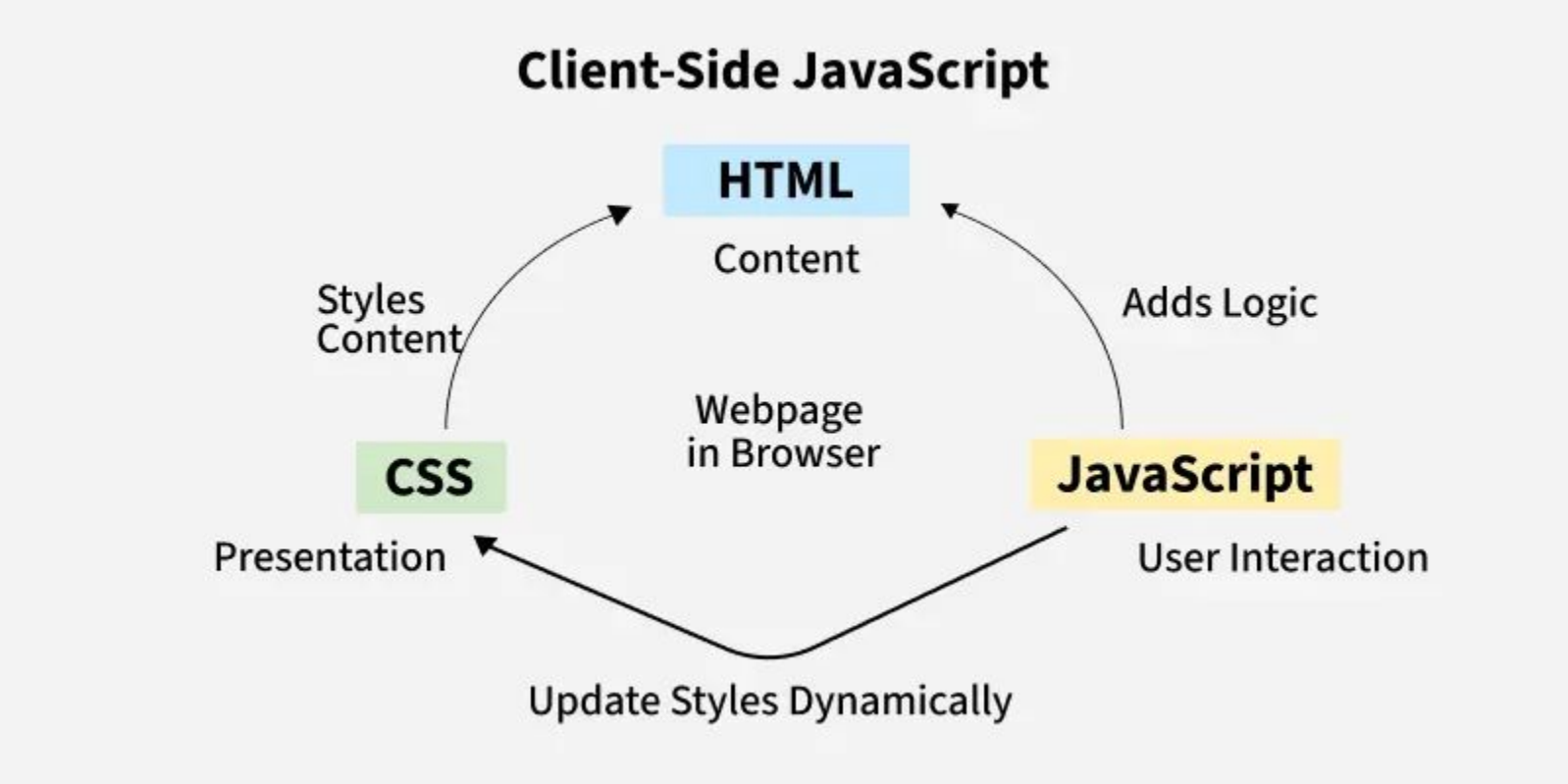
Since 2016, versions are named by year (**ECMAScript 2016, 2017, 2018**, ...)




TOP
10 JAVASCRIPT FRAMEWORKS
ANGULARJS
by Google
React
Vue
ember
RIOT
METEOR
aurelia
BACKBONEJS
polymer
MERCURY
networks
www.konstantinfo.com



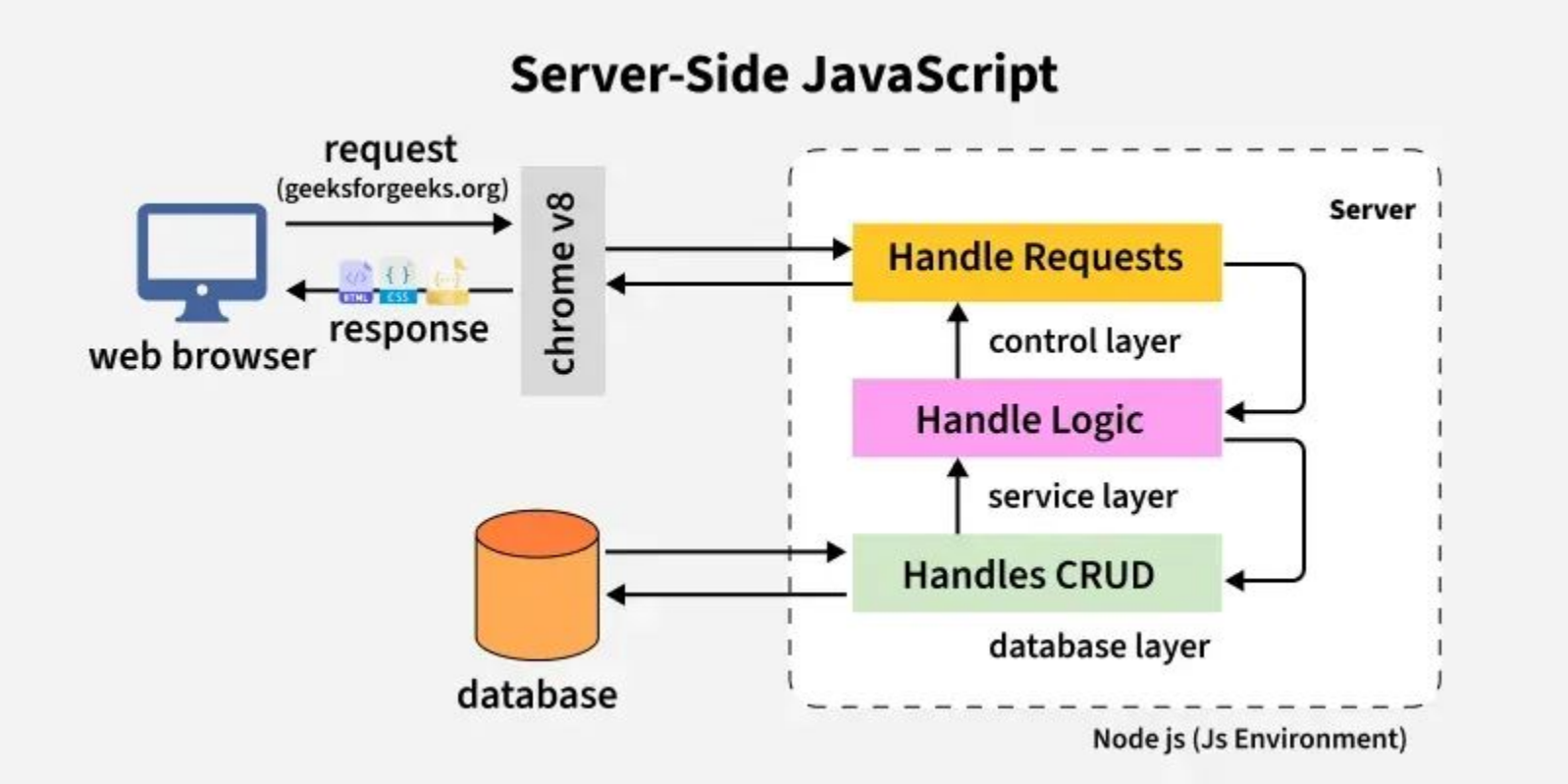
JavaScript's flexibility extends to both the client-side and server-side, allowing developers to create complete web



update and change both **HTML** and **CSS**

calculate, manipulate and validate **data**

- Change HTML Content
- Change HTML Attribute Values
- Change HTML Styles (CSS)
- Hide and show HTML Elements



Access databases

File handling

Security management

Send responses

JavaScript insertion

Inline JavaScript

You can write JavaScript code directly inside the HTML element using the onclick, onmouseover, or other event handler attributes

```
<button onclick="alert('Button Clicked!')">
    Click Here
</button>
```

JavaScript insertion

Internal JavaScript

In HTML, JavaScript code is inserted between <script> and </script> tags

Scripts can be placed in the <body>, or in the <head> section

<script> </script>

head

body

```
<html>
    <head>
        <script>document.getElementById("demo1").innerHTML = "Paragraph changed.";</script>
    </head>
    <body>
        <script>document.getElementById("demo2").innerHTML = "Paragraph changed.";</script>
    </body>
</html>
```

JavaScript insertion

External JavaScript

- Scripts can also be placed in external files
- It separates HTML and code makes HTML and JavaScript easier to read and maintain
- Cached JavaScript files can speed up page loads

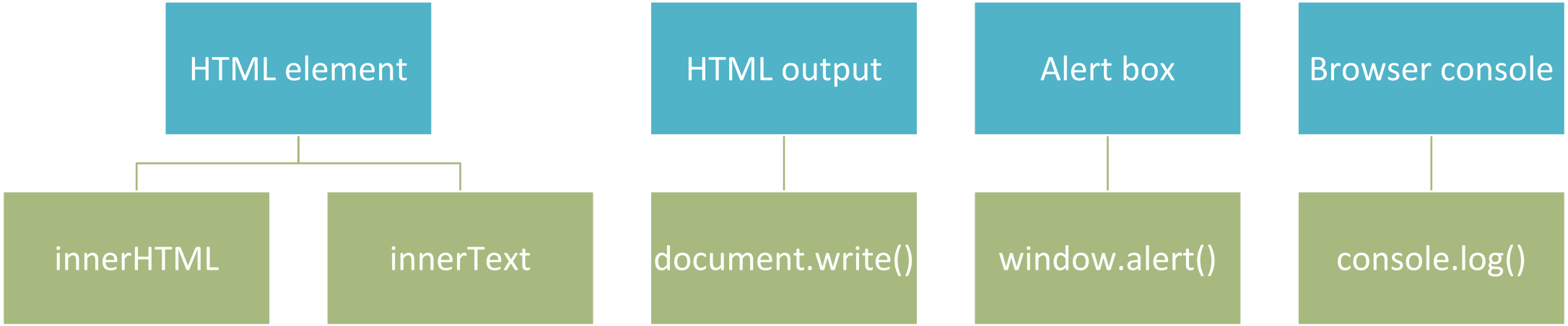
```
<script src="myScript.js"></script
>
```

External Files

myScript.js

```
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph
changed.";
}
```

JavaScript Display



JavaScript Display - HTML Element

use the document.getElementById(id) method to access an HTML element

innerHTML

innnerText

```
<p id="demo1"></p>
<p id="demo2"></p>
<script>
    document.getElementById("demo1").innerHTML = "<h1>Hello World</h1>";

    document.getElementById("demo2").innerText = "<h1>Hello World</h1>";
</script>
```



JavaScript Display – HTML output

document.write( )

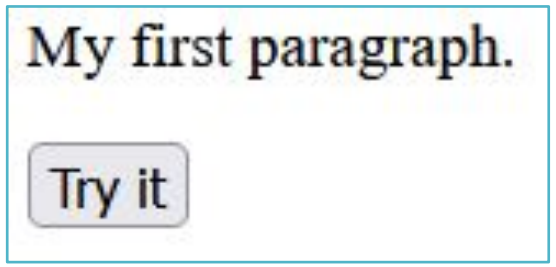
Between page load

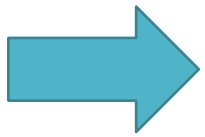
```
<p>
    result of 1+1 is <script>document.write(1 + 1);</script>
</p>
```

result of 1+1 is 2

After page loaded

```
<body>
    <p>My first paragraph.</p>
    <button type="button" onclick="document.write(5 + 6)">Try it</button>
</body>
```





11

JavaScript Display – Alert box

window.alert( )

alert()

```
<button type="button" onclick="window.alert(5 + 6);">click</button>
```

Alert on click button



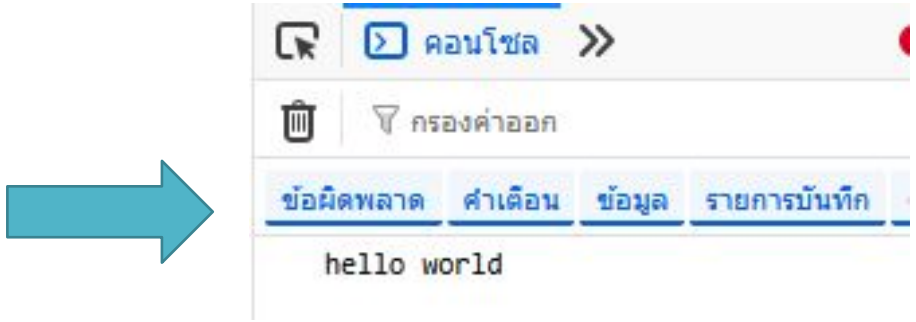
Alert on load page

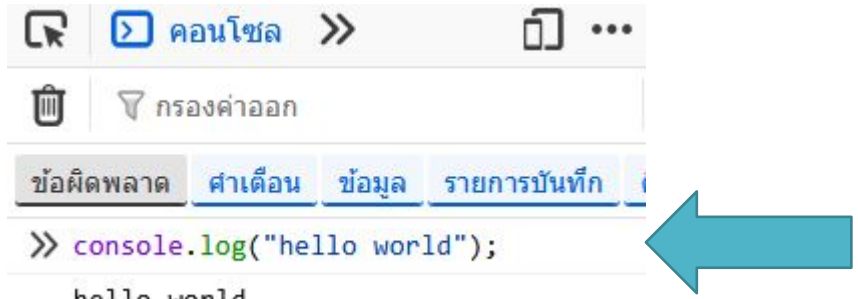
```
    <p>Hello world</p>
    <button type="button">click</button>
    <script>
        window.alert(5+6);
    </script>
```

JavaScript Display – Browser console

console.log()

```
<script>
    console.log("hello world");
</script>
```





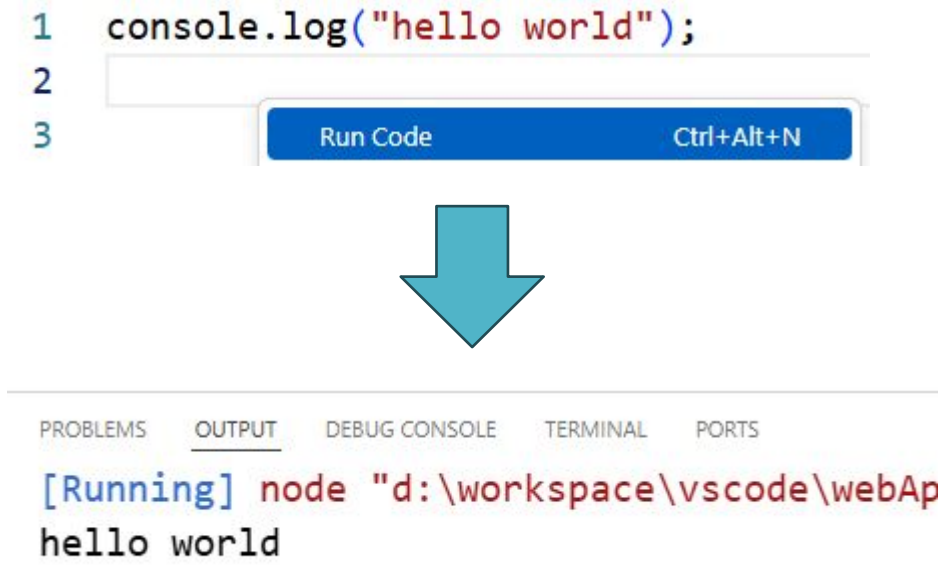
Command line

JavaScript Display – VSCode

Install







Statements

Semicolons ; - Add a semicolon at the end of each executable statement

White Space - You can add white space to your script to make it more readable.

```
let x = y + z;
```

✓

Line Length and Line Breaks - avoid code lines longer than 80 characters

the best place to break it is after an operator

```
document.getElementById("demo").innerHTML =
"Hello Dolly!";
```

✓

Comments /* */ and //

```
/*comment*/
let Name, name; //comment
```

✓

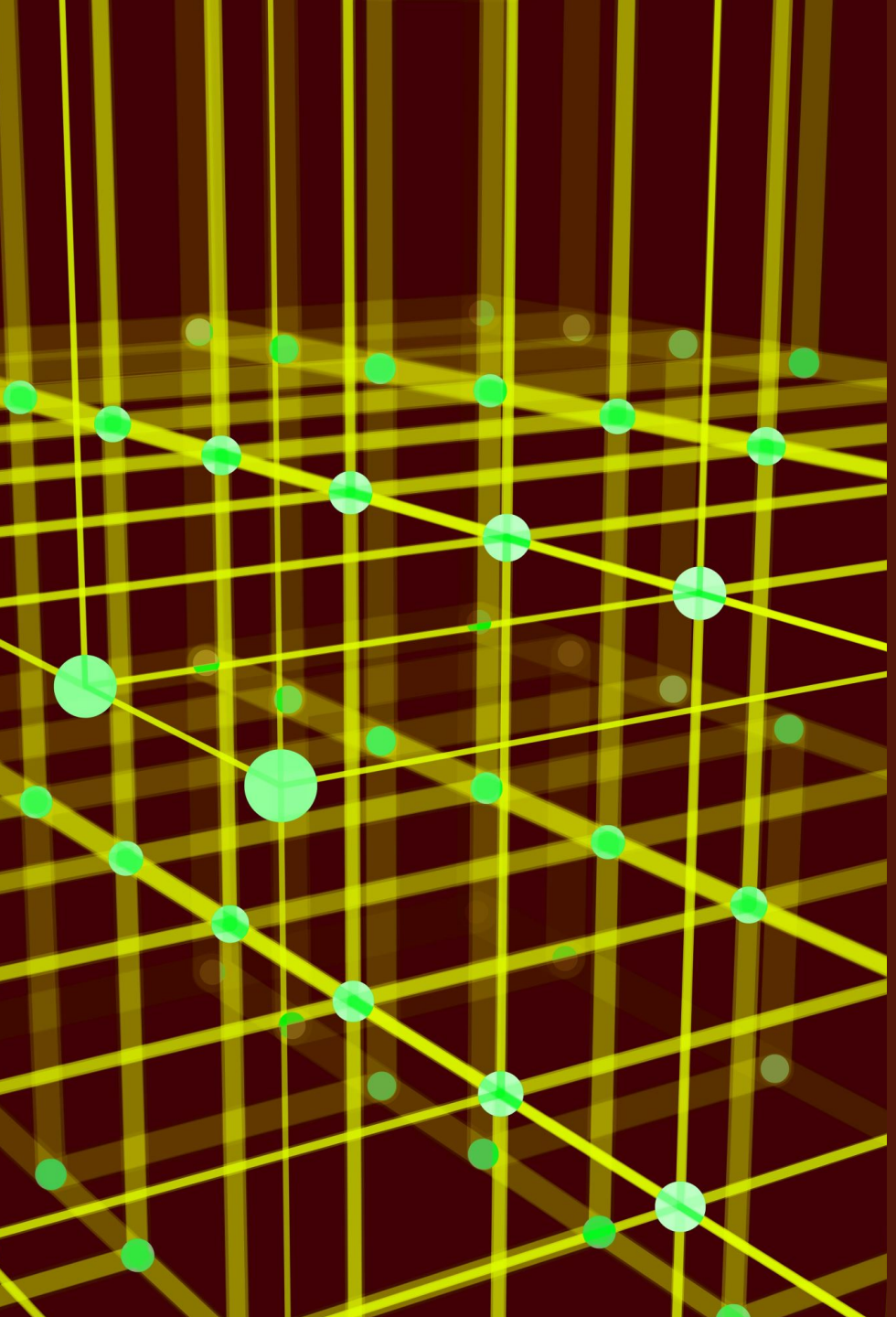
Case Sensitive

```
Name = "John";
name = "Doe";
```

✓

Keywords

Keyword	Description
var	Declares a variable
let	Declares a block variable
const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function
return	Exits a function
try	Implements error handling to a block of statements



Variables

Variables

var

let

const

declaration	description	example
automatically	automatically declared when first used	x=5;
var	used in code written for older browsers	var x = 5;
let	For currently browsers	let x = 5;
const	Type and value should not be changed	const x = 5;

Variables

VAR

Variables declared with var have Global Scope

Does not have to be declared

Can be redeclared

Used in code written for older browsers

LET / CONST

Variables declared with let have Block Scope

Must be Declared before use

Cannot be Redeclared in the same scope

added to JavaScript in 2015

const cannot be Reassigned



```
{
  var x = 2;
}
// x CAN be used here
```

Global Scope



```
{
  let x = 2;
}
// x can NOT be used here
```

Block Scope



```
var x = "John Doe";

var x = 0;
```

Redeclared



```
let x = "John Doe";

let x = 0;
```

Cannot be redeclared in same scope



```
var x = 10;
// Here x is 10

{
var x = 2;
// Here x is 2
}

// Here x is 2
```



```
let x = 10;
// Here x is 10

{
let x = 2;
// Here x is 2
}

// Here x is 10
```

const cannot be Reassigned

✓
```
const PI = 3.14159265359;
```

✗
```
const PI;
PI = 3.14159265359;
```

✗
```
const PI = 3.141592653589793;
PI = 3.14;      // error
PI = PI + 10;   // error
```

```
Uncaught TypeError: Assignment to constant variable.
```

const with block scope

✓
```
const x = 2;

{
  const x = 3;
}

{
  const x = 4;
}
```

Check 1

```
var x = 2;
const x = 3;
```

x =

```
const x = 2;
var x = 3;
```

x =

```
const x = 2;
{
var x = 3;
}
```

x =

```
var x = 2;
{
const x = 3;
}
```

x =

```
const x = 2;
{
let x = 3;
}
```

x =

```
var x = 2;
let x = 3;
```

x =

```
let x = 2;
var x = 3;
```

x =

```
var x = 2;
{
let x = 3;
const x = 4;
}
```

x =

```
var x = 2;
{
let x = 3;
}
{
const x = 4;
}
```

x =

Variable - Const

const cannot be Redeclared

const cannot be Reassigned

const have Block Scope

Use const when you declare:

- A new Array
- A new Object
- A new Function
- A new RegExp



It defines a constant **reference** to a value

const:
Change the elements of constant array
Change the properties of constant object



```
const cars = ["Saab", "Volvo", "BMW"];
cars[0]      = "Toyota";  // change an element:
cars.push("Audi"); // add an element:
cars    = ["Toyota", "Volvo"];    // ERROR
```



reassign

Variables - Naming

The general rules for constructing names for variables are:

- can contain letters, digits, underscores, and dollar signs.
- must begin with a letter or $ or _
- case sensitive
- cannot use reserved words

```
let x1 = 5;
```


```
let _x = 5;
```


```
let $$x = 5;
```


```
let 1x = 5;
```


Underscore

- first_name
- last_name

Upper Camel Case

- FirstName
- LastName

Lower Camel Case

- firstName
- lastName

Variable Declaration

Declaration and assign value

```
let name = "John";
const x = 5, y = 10;
z = 30;
```

```
let person = "John Doe",
carName = 'Volvo',
price = 200;
```

Declaration without assign value

```
let name, lastName;
const age;
```

the variable has no value or undefined.

```
<script>
  let carName;
  document.getElementById("demo").innerHTML = carName;
</script>
```



undefined

Re-declaration

cannot re-declare a variable declared with let or const



Operator

Operators - Basic operators

arithmetic operators 

assignment operator 

```
var x, y;
x = 5 + 3;
y = x * 2;
```

concatenate string 

```
fullName="John" + " " +
"Doe"
```

Operators - Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation
/	Division
%	Modulus
++	Increment
--	Decrement

x++ / ++x

x-- / --x

Operators - Assignment Operators

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

Operators - Logical Operators

Operator	Description	Example
&&	logical AND	(x < 10 && y > 1) is true
||	logical OR	(x == 5 || y == 5) is false
!	logical NOT	!(x == y) is true

Operators - Logical Assignment Operators

Operator	Definition	Definition	Example
&&=	Logical AND assignment operator	If the first value is **true**, the second value is assigned.	x &&= 10
||=	Logical OR assignment operator	If the first value is **false**, the second value is assigned	x ||= 10
??=	Nullish coalescing assignment operator	If the first value is **undefined** or **null**, the second value is assigned	x ??= 10

Check 2

```
let x = true;
x &&= 10;
```

x = 10

```
let x = true;
x ||= 10;
```

x =

```
let x = true;
x ??= 10;
```

x =

```
let x = false;
x &&= 10;
```

x = false

```
let x = false;
x ||= 10;
```

x =

```
let x = false;
x ??= 10;
```

x =

```
let x = 1;
x &&= 10;
```

x = 10

```
let x = 1;
x ||= 10;
```

x =

```
let x = 1;
x ??= 10;
```

x =

```
let x = 0;
x &&= 10;
```

x = 0

```
let x = 0;
x ||= 10;
```

x =

```
let x = 0;
x ??= 10;
```

x =

```
let x;
x &&= 10;
```

x = undefined

```
let x;
x &&= 10;
```

x =

```
let x;
x ??= 10;
```

x =

```
let x = null;
x &&= 10;
```

x = null

```
let x = null;
x &&= 10;
```

x =

```
let x = null;
x ??= 10;
```

x =

Operators - Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator **condition ? exprIfTrue : exprIfFalse;**

ternary operator

x > y ? max = x : max = y

max = x > y ? x : y

Ternary operator example

```
var userType;
if (userIsYoungerThan18) {
  userType = "Minor";
} else {
  userType = "Adult";
}
```

condition ? exprIfTrue : exprIfFalse;

```
var userType = userIsYoungerThan18 ? "Minor" : "Adult";
```

```
if (userIsYoungerThan21) {
  serveDrink("Grape Juice");
} else {
  serveDrink("Wine");
}
```

```
serveDrink(userIsYoungerThan21 ? "Grape Juice" :
"Wine");
```

```
if (userIsYoungerThan21) {
 serveGrapeJuice();
} else {
 serveWine();
}
```

```
userIsYoungerThan21 ? serveGrapeJuice() : serveWine();
```

Nested ternary operators

```
serveDrink(userIsYoungerThan4 ? 'Milk' : userIsYoungerThan21 ? 'Grape Juice' :
'Wine');
```

Check 3

```
const score = 80;
let scoreRating;

if (score > 70) {
  scoreRating = "Excellent“;
} else {
  scoreRating = "Do better“;
}
```

let scoreRating =

```
const score = 80;
let scoreRating;

if (score > 70) {
  scoreRating = "Excellent“;
} else {
  if (score > 50) {
    scoreRating = “Good“;
  } else {
    scoreRating = "Do better“;
  }
}
```

let scoreRating =

Operators

Type Operators

Operator	Description
typeof	Returns the type of a variable
instanceof	Returns true if an object is an **instance of an object type**

```
typeof "John Doe"      // Returns "string"
typeof 0               // Returns "number"
typeof 314             // Returns "number"
```

```
function Car(make, model, year) {
    this.make = make;
    this.model = model;
    this.year = year;
}
const auto = new Car("Honda", "Accord", 1998);

console.log(auto instanceof Car); // true
```

Operators

Bitwise Operators

Note : JavaScript uses 32-bit signed numbers

Operator	Description	Example	Same as	Result	Decimal
&	AND	5 & 1	0101 & 0001	0001	1
|	OR	5 | 1	0101 | 0001	0101	5
~	NOT	~ 5	~0101	1010	10
^	XOR	5 ^ 1	0101 ^ 0001	0100	4
<<	left shift	5 << 1	0101 << 1	1010	10
>>	right shift	5 >> 1	0101 >> 1	0010	2
>>>	unsigned right shift	5 >>> 1	0101 >>> 1	0010	2

Operators - String Operators

Operator	Description
== != > >= < <=	String Comparison strings are compared alphabetically
+	concatenate strings
+=	concatenate strings and assign value

Check 4

name = ?

```
var name = "John";
var name;
```

```
let name = "John";
let name;
```

```
const name = "John";
const name;
```

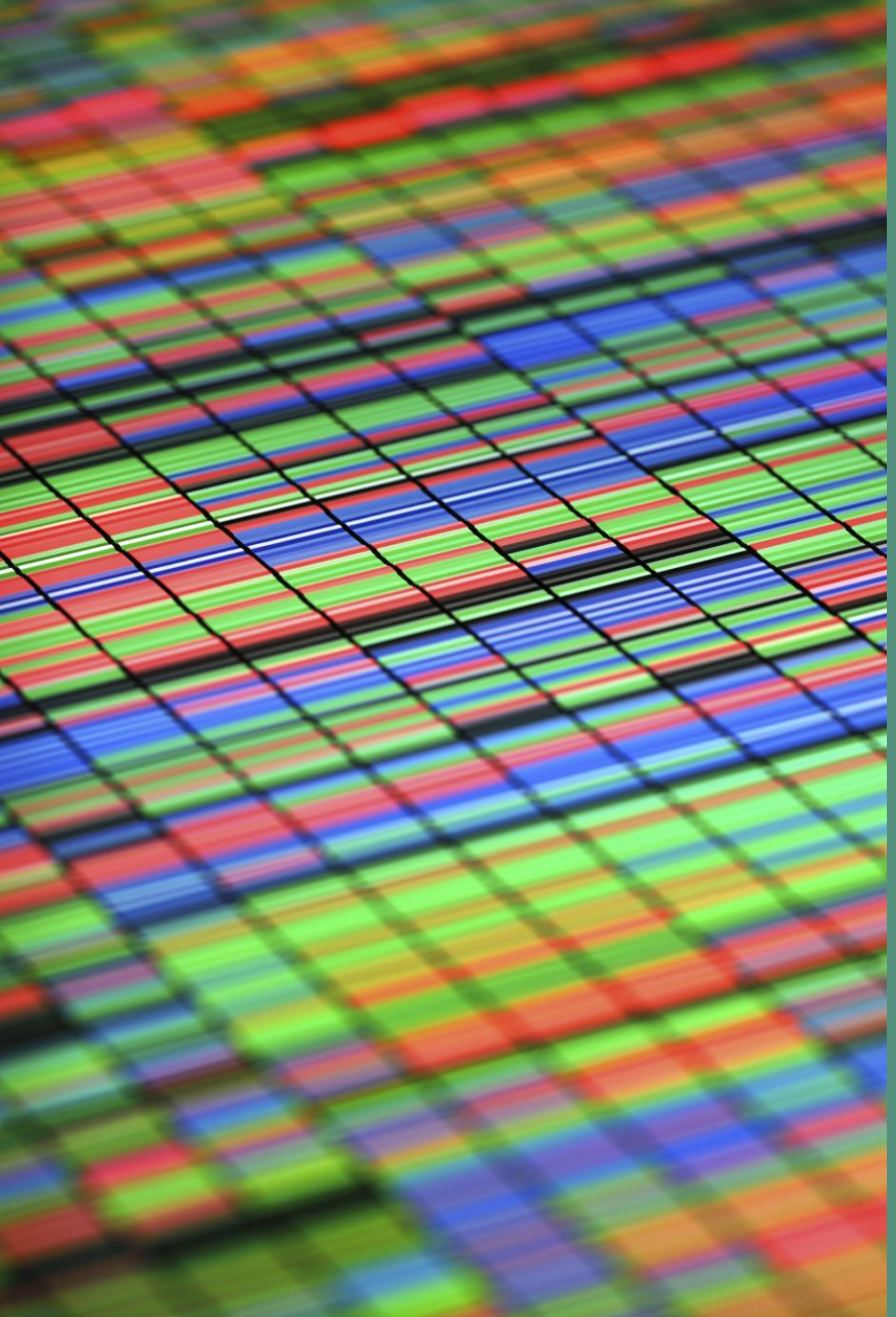
```
let name;
```

x = ?

```
let x = "5" + 2 + 3;
```

```
let x = 2 + 3 + "5";
```

```
let x = 5 + 5 + "5" + 5 + 5 + "5";
```



Data types

Data Types

String

- let name="John“;
- let lastName = 'Doe';

Number

- let length = 16;
- let weight = 7.5;

Bigint

- let x = BigInt("123456789012345678901234567890");

Boolean

- let x = true;
- let y = false;

Array

- const cars = ["Saab", "Volvo", "BMW"];

Object

- const date = new Date("2022-03-25");
- const person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};

Undefined

- let car;
- name = undefined;

Null

- let name=null;

undefined means Value is undefined, type is undefined

Javascript numbers are always one type: double (64-bit floating point)

Number

Integer

Floating point

the number type represents numeric values

```
// integer value
let integer_number = -3;
```

```
// floating-point value
let float_number = 3.15;
```

```
//scientific notation
let x = 123e5;    // 12300000
let y = 123e-5;   // 0.00123
```

```
// Hexadecimal
let x = 0xFF; //255
```

Floating point arithmetic is not always 100% accurate:

```
let x = 0.2 + 0.1;
console.log(x);
```

```
0.30000000000000004
```

```
let x = (0.2 * 10 + 0.1 * 10) / 10;
console.log(x);
```

```
0.3
```

Check 5

Number

NAN - NOT A NUMBER

indicating that a number is not a legal number.

```
let x = 100 / "Apple";
console.log(x);
```

NaN

```
let x = 100 / "10";
console.log(x);
```

10

```
console.log(typeof NaN);
```

ISNAN()

isNaN() find out if a value is a not a number

```
let x = 100 / "Apple";
console.log (isNaN(x));
```

```
let x = NaN;
let y = 5;
let z = x + y;
console.log(z);
```

```
let x = NaN;
let y = "5";
let z = x + y;
console.log(z);
```

Big Int

BigInt is a type of number that can represent very large or very small integers

Supported in a newer version of JavaScript (ES11)

appending n to the end of an integer

```
// BigInt value
let value1 = 90071992512474099 8n;

// add two big integers
let result1 = value1 + 1n;

console.log(result1);
```

```
90071992512474099 9n
```

```
// add big integers with integer
let result1 = value1 + 1;
```

```
error
```



Strings

Strings

A JavaScript string is zero or more characters written inside quotes

```
let text1 = "He is called 'Johnny'"; // Double quotes
let text2 = 'He is called "Johnny"'; // Single quotes
```

backslash escape character (\)

Code	Result	Description
\'	'	Single quote
\"	"	Double quote
\\	\	Backslash

```
let text3 = 'He is called \'Johnny\''; // single quote
let text4 = "He is called \"Johnny\""; // double quote
let text5 = "He is called \\Johnny\\"; // backslash
```

Strings - String Templates

use back-ticks (``) rather than the quotes ("") to define a string

```
let text1 = `He's often called "Johnny"`; //Template Strings
```

allow multiline strings:

```
<pre id="demo"></pre>

<script>
let text =
`The quick
brown fox
jumps over
the lazy dog`;
document.getElementById("demo").innerHTML = text;
</script>
```



```
The quick
brown fox
jumps over
the lazy dog
```

allow variables in strings.

```
let firstName = "John";
let lastName = "Doe";

let text = `Welcome ${firstName}, ${lastName}!`;
```

interpolation of expressions in strings

```
let price = 10;
let VAT = 0.25;

let total = `Total: ${(price * (1 + VAT)).toFixed(2)}`;
```



objects

Objects

Objects are containers for Properties and Methods.

Properties are named Values.

Methods are Functions stored as Properties.

Properties can be primitive values, functions, or even other objects.

CAR



Properties	Methods
• Name • Model • color	• Start() • Drive() • Break() • Stop()

Objects

All JavaScript values, except primitives, are objects

primitive value
• string • number • boolean • null • undefined • symbol • bigint

objects
• Objects • Maths • Functions • Dates • Arrays • Maps • Sets

Objects

An object literal is a list of **key : value** pairs inside curly braces **{ }**

```
const person = {
firstName:"John",
lastName:"Doe",
age:50,
eyeColor:"blue"};
```

```
const person = {};
// Add Properties
person.firstName = "John";
person.lastName = "Doe";
person.age = 50;
person.eyeColor = "blue";
```

```
const person = new Object({
person.firstName = "John";
person.lastName = "Doe";
person.age = 50;
person.eyeColor = "blue";
});
```

```
const person = new Object();
// Add Properties
person.firstName = "John";
person.lastName = "Doe";
person.age = 50;
person.eyeColor = "blue";
```

declare objects with the `const` keyword

Objects

JavaScript Objects are addressed by reference, not by value.

```
//Create an Object
const person = {
  firstName:"John",
  lastName:"Doe",
  age:50, eyeColor:"blue"
}

// Create a copy
const x = person;

// This will change age in person:
x.age = 10;
```



```
person.age = 10;
```

Objects - properties

Accessing Object Properties

```
objectName.propertyName
```



```
let lName = person.lastName;
```

```
objectName["propertyName"]
```



```
let lName = person["lastName"];
```

```
objectName[expression]
```



```
Let x ="lastName";
let lName = person[x];
```

Adding New Properties

```
person.nationality = "English";
```

Deleting Properties

```
delete person.age;
delete person["age"];
```

Objects - method

Object Methods

Methods are actions that can be performed on objects.

Methods are function definitions stored as property values.

property : **function()** {return … ;}

objectName.methodName()

```
const person = {
  firstName: "John",
  lastName : "Doe",
  id       : 5566,
  fullName : function() {
    return this.firstName + " " + this.lastName;
  }
};
```

```
name = person.fullName();
```

this refers to the current object

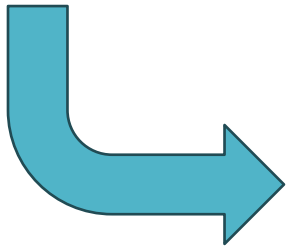
fullName return the function definition

Objects - constructor

To create an **object type** we use an **object constructor function**.

name constructor functions with an upper-case first letter

```
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}
```



```
const myFather = new Person("John", "Doe", 50, "blue");
const myMother = new Person("Sally", "Rally", 48, "green");
const mySister = new Person("Anna", "Rally", 18, "green");
const mySelf = new Person("Johnny", "Rally", 22, "green");
```

Objects - Nested Objects

Property values in an object can be other objects

```
myObj = {
  name:"John",
  age:30,
  myCars: {
    car1:"Ford",
    car2:"BMW",
    car3:"Fiat"
  }
}
```

access nested objects using the dot notation or the bracket notation

```
myObj.myCars.car2;
```

```
myObj.myCars["car2"];
```

```
myObj["myCars"]["car2"];
```

```
let p1 = "myCars";
let p2 = "car2";
myObj[p1][p2];
```

Objects - Display

```
const person = {
    firstName: "John",
    lastName: "Doe",
    id: 5566,
    fullName: function () {
        return firstName + " " + lastName;
    }
};
```

```
document.getElementById("demo").innerHTML = person;
```



[object Object]

```
document.getElementById("demo").innerHTML =
person.firstName + " " + person.lastName;
```



John Doe

```
console.log(person);
```



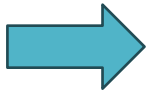
Object { firstName: "John", lastName: "Doe", id: 5566, fullName: fullName() }

`console.log()`, the of the objects display its contents in a human-readable and often interactive format

Objects - Display

Using Object.values()

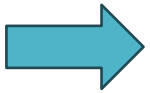
```
let text = Object.values(person);
```



John,Doe,5566,function () { return firstName + " " + lastName; }

Using JSON.stringify()

```
let text = JSON.stringify(person);
```



{"firstName":"John","lastName":"Doe","id":5566}

Objects - Display

Displaying Properties in a Loop

```
// Build a Text
let text = "";
for (let x in person) {
  text += person[x] + " ";
};
```



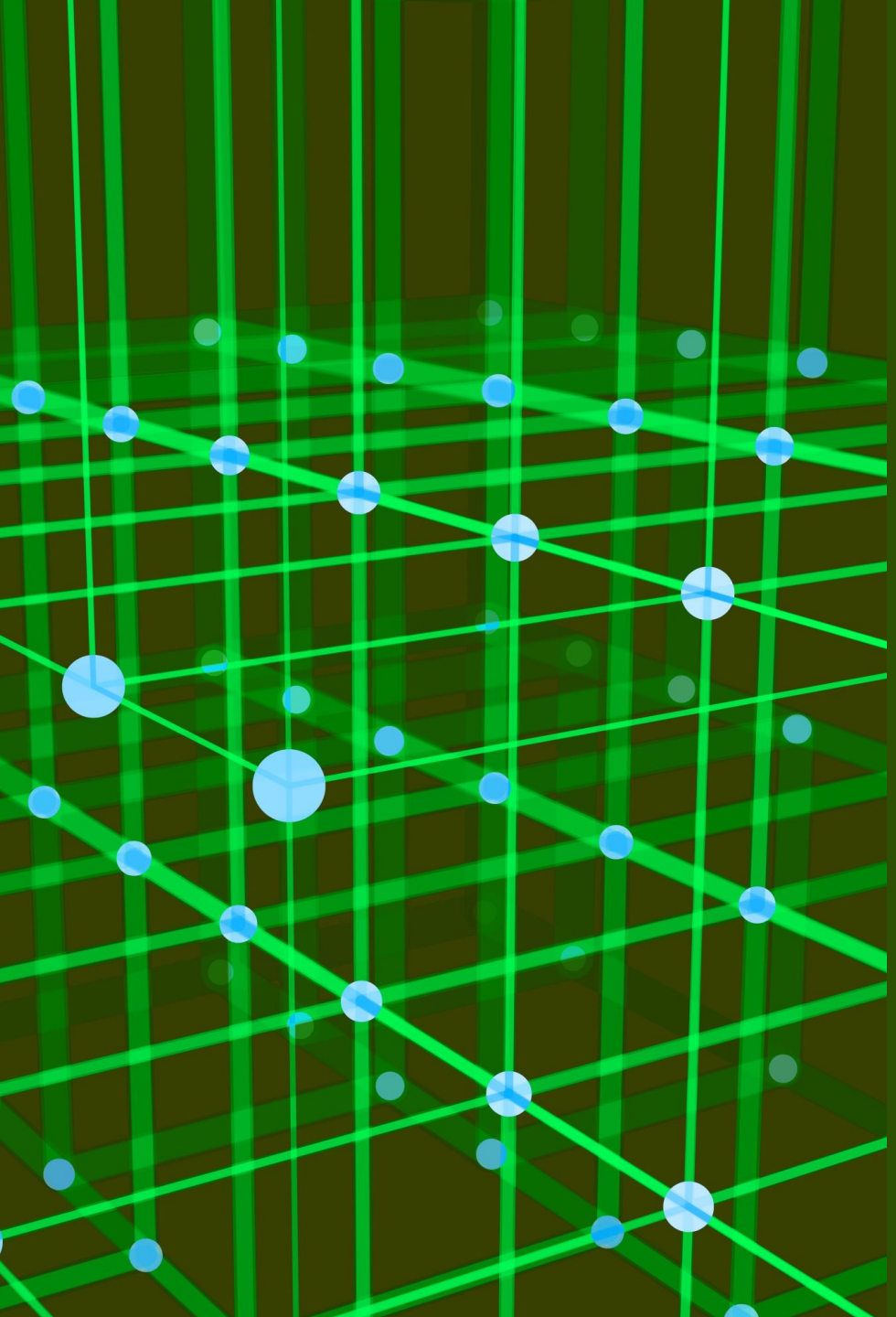
John Doe 5566 function () { return firstName + " " + lastName; }

Using Object.entries()

```
let text = "";
for (let [key1, value1] of Object.entries(person)) {
    text += key1 + " : " + value1 + ", "
}
```



firstName : John, lastName : Doe, id : 5566, fullName : function () { return firstName + " " + lastName; },



Arrays

Arrays

Creating an Array

```
const array_name = [item1, item2, ...];
```

```
const cars = ["Saab", "Volvo", "BMW"];
```

```
const cars = [];
cars[0]= "Saab";
cars[1]= "Volvo";
cars[2]= "BMW";
```

```
const array_name = new Array();
```

```
const cars = new Array();
```

```
const cars = new Array("Saab", "Volvo", "BMW");
```

```
const cars = new Array(5);   // Create 5 undefined elements
```



Accessing Array Elements

```
let car = cars[0];
```

Changing an Array Element

```
cars[0] = "Opel";
```

In JavaScript, **arrays** always use **numbered indexes**

```
const person = [];
person["firstName"] = "John";
person["lastName"] = "Doe";
person["age"] = 46;
```



Arrays - elements

Array Elements Can Be Difference Type

```
const person = ["John", "Doe", 46];
```

Array Elements Can Be Objects Functions and Arrays

```
const person = ["John",  // string
["Orange", "Apple", "Mango"], // array
(name) => { return "hello "+ name; }, // function
{ address: "Thailand", age: 35 } // object
];
document.getElementById("demo").innerHTML = person[2]("John");
```



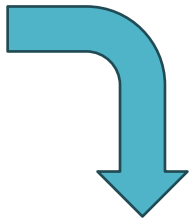
Arrays – Array to string

Converting an Array to a String

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
document.getElementById("demo").innerHTML = fruits.toString();
```

```
document.getElementById("demo").innerHTML = fruits;
```



Banana,Orange,Apple,Mango

Arrays – Properties and methods

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];

fruits.length; // array length

fruits[0] // first element

fruits[fruits.length - 1] // last element

fruits.push("Lemon"); // Adding Array Elements

fruits[fruits.length] = "Lemon"; // Adding Array Elements
```

```
let type = typeof fruits;
```



object

```
Array.isArray(fruits)
```



true

```
fruits instanceof Array
```

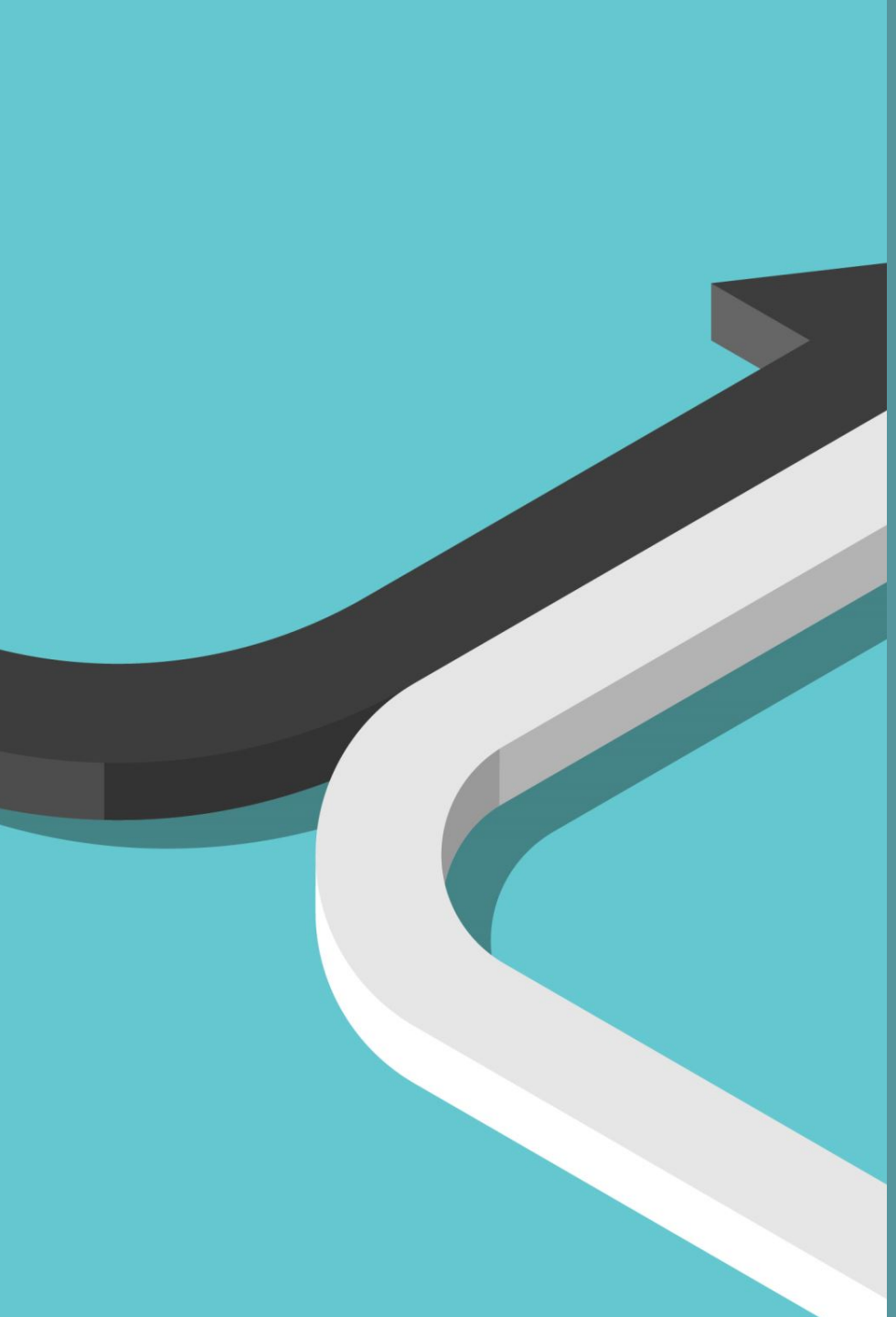


true

Arrays – Nested Arrays and Objects

Values in objects can be arrays, and values in arrays can be objects

```
const myObj = {
  name: "John",
  age: 30,
  cars: [
    {name:"Ford", models:["Fiesta", "Focus", "Mustang"]},
    {name:"BMW", models:["320", "X3", "X5"]},
    {name:"Fiat", models:["500", "Panda"]}
  ]
}
```



Control flow

If else

```
if (condition) {
  //  block of code
}
```

```
if (hour < 18) {
  greeting = "Good day";
}
```

```
if (condition) {
  //  block of code
} else {
  //  block of code
}
```

```
if (hour < 18) {
  greeting = "Good day";
} else {
  greeting = "Good evening";
}
```

```
if (condition1) {
  //  block of code
} else if (condition2) {
  //  block of code
} else {
  //  block of code
}
```

```
if (time < 10) {
  greeting = "Good morning";
} else if (time < 20) {
  greeting = "Good day";
} else {
  greeting = "Good evening";
}
```

switch

```
switch(expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
    // code block
}
```

```
switch (new Date().getDay()) {
  case 0:
    day = "Sunday";
    break;
  case 1:
    day = "Monday";
    break;
  case 2:
     day = "Tuesday";
    break;
  case 3:
    day = "Wednesday";
    break;
  case 4:
    day = "Thursday";
    break;
  case 5:
    day = "Friday";
    break;
  case 6:
    day = "Saturday";
  default:
    day = "Unknown";
}
```

```
switch (new Date().getDay()) {
  case 4:
  case 5:
    text = "Soon it is Weekend";
    break;
  case 0:
  case 6:
    text = "It is Weekend";
    break;
  default:
    text = "Looking forward to the Weekend";
}
```

Loop

JavaScript supports different kinds of loops

for - loops through a block of code a number of times
for/in - loops through the properties of an object
for/of - loops through the values of any iterable
while - loops through a block of code while a specified condition is true
do/while - also loops through a block of code while a specified condition is true

Loop - for

```
for (expression; condition; expression) {
  // code block to be executed
}
```

```
for (let i = 0; i < 5; i++) {
  text += "The number is " + i + "<br>";
}
```

Loop Scope

Using var in a loop:

```
var i = 5;

for (var i = 0; i < 10; i++) {
  // some code
}

// Here i is 10
```

Using let in a loop:

```
let i = 5;

for (let i = 0; i < 10; i++) {
  // some code
}

// Here i is 5
```

Loop – while / do-while

```
while (condition) {
  // code block to be executed
}
```

```
while (i < 10) {
  text += "The number is " + i;
  i++;
}
```

```
do {
  // code block to be executed
}
while (condition);
```

```
do {
  text += "The number is " + i;
  i++;
}
while (i < 10);
```

```
const cars = ["BMW", "Volvo", "Saab", "Ford"];
```

For loop - Array

```
let i = 0;
let text = "";

for (;cars[i]; i++) {
  text += cars[i];
}
```

While loop - Array

```
let i = 0;
let text = "";

while (cars[i]) {
  text += cars[i];
  i++;
}
```

Loop – for-in

```
for (property in object)
    statement
```

```
const person = { name: "John", lastName: "Doe", age: 35 };

for (const property in person) {
    document.write(`${property} : ${person[property]}<br>`);
}
```

name : John
lastName : Doe
age : 35

```
const array = [10, 20, 30];

for (const value in array) {
    document.write(array[value] + "<br>");
}
```

10
20
30

Loop – for-of

```
for (variable of iterable)
  statement
```

```
const person = { name: "John", lastName: "Doe", age: 35 };

for (let [key, val] of Object.entries(person)) {
    document.write (`${key} : ${val}<br>`);
}
```

name : John
lastName : Doe
age : 35

```
const array = [10, 20, 30];
for (const value of array) {
      document.write(value+"<br>");
}
```

10
20
30

Break and Continue

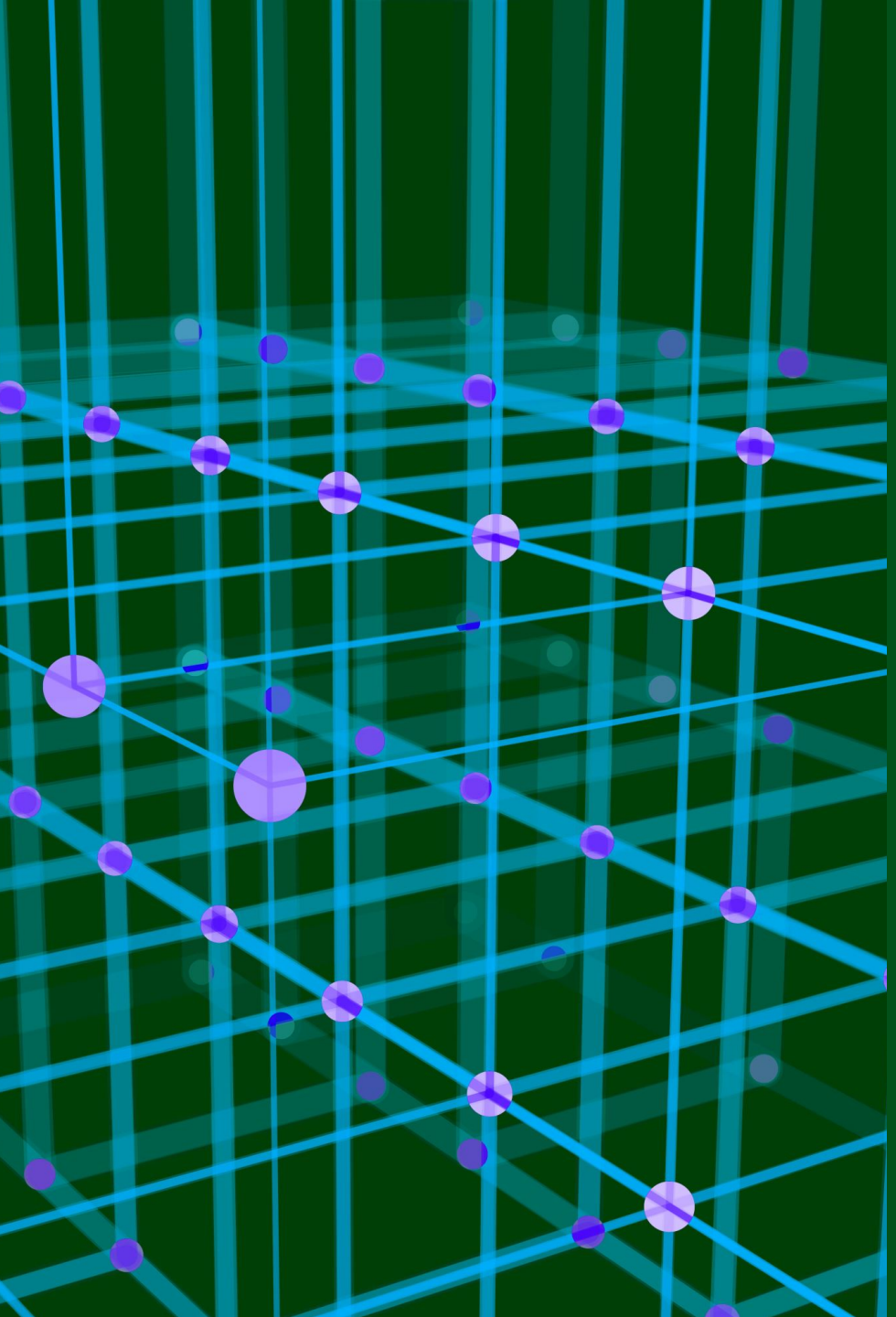
The break statement terminates the execution of a loop or a switch statement.

The continue statement skips the current iteration in a loop and proceed to the next iteration.

```
for (let i = 0; i < 10; i++) {
  if (i === 3) { break; }
  text += "The number is " + i + "<br>";
}
```

```
for (let i = 0; i < 10; i++) {
  if (i === 3) { continue; }
  text += "The number is " + i + "<br>";
}
```

```
switch (new Date().getDay()) {
  case 4:
  case 5:
    text = "Soon it is Weekend";
    break;
  case 0:
  case 6:
    text = "It is Weekend";
    break;
  default:
    text = "Looking forward to the Weekend";
}
```



Functions

Functions

A JavaScript function is a block of code designed to perform a particular task

```
function name(parameter1, parameter2) {
  // code to be executed
}
```

```
function myFunction(a, b) {
  return a * b;
}
```

Variables declared within a JavaScript function, become LOCAL to the function.

```
// code here can NOT use carName
function myFunction() {
  let carName = "Volvo";
  // code here CAN use carName
}
// code here can NOT use carName
```

```
let value = myFunction(4, 3);
```



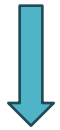
12

```
let value = myFunction();
```



NaN

```
let value = myFunction;
```



```
function myFunction(a, b) { return a * b; }
```

Functions - Arrow Functions

Arrow functions must be defined **before** they are used

Arrow functions allow us to write shorter function syntax:

```
let name = (parameter1, parameter2) => {   }
```

```
function myFunction(a, b) {
  return a * b;
}
```



```
let myFunction = function (a, b) { return a * b; }
```



```
let myFunction = (a, b) => { return a * b; }
```

```
let hello = () => { return "Hello World!"; }
```


```
let hello = () => "Hello World!";
```


```
let hello = (val) => "Hello " + val;
```


```
let hello = val => "Hello " + val;
```


```
let myFunction = (a, b) => { return a * b; }
```


```
let myFunction = (a, b) => return a * b;
```


```
let myFunction = (a, b) => a * b;
```


```
let myFunction = (a, b) => {a * b;}
```




Events

Events

HTML events are "things" that happen to HTML elements

can be something the browser does, or something a user does

HTML allows event handler attributes, with JavaScript code, to be added to HTML elements

```
<element event='some JavaScript'>
```

```
<button onclick="this.innerHTML=Date()">The time is?</button>
```

```
<button onclick="document.getElementById('demo').innerHTML = Date()">
The time is?</button>
```

```
<button onclick="displayDate()">The time is?</button>
```

```
<script>

   ____________________________

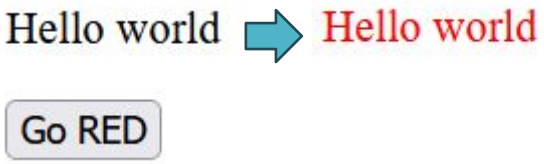
</script>
```

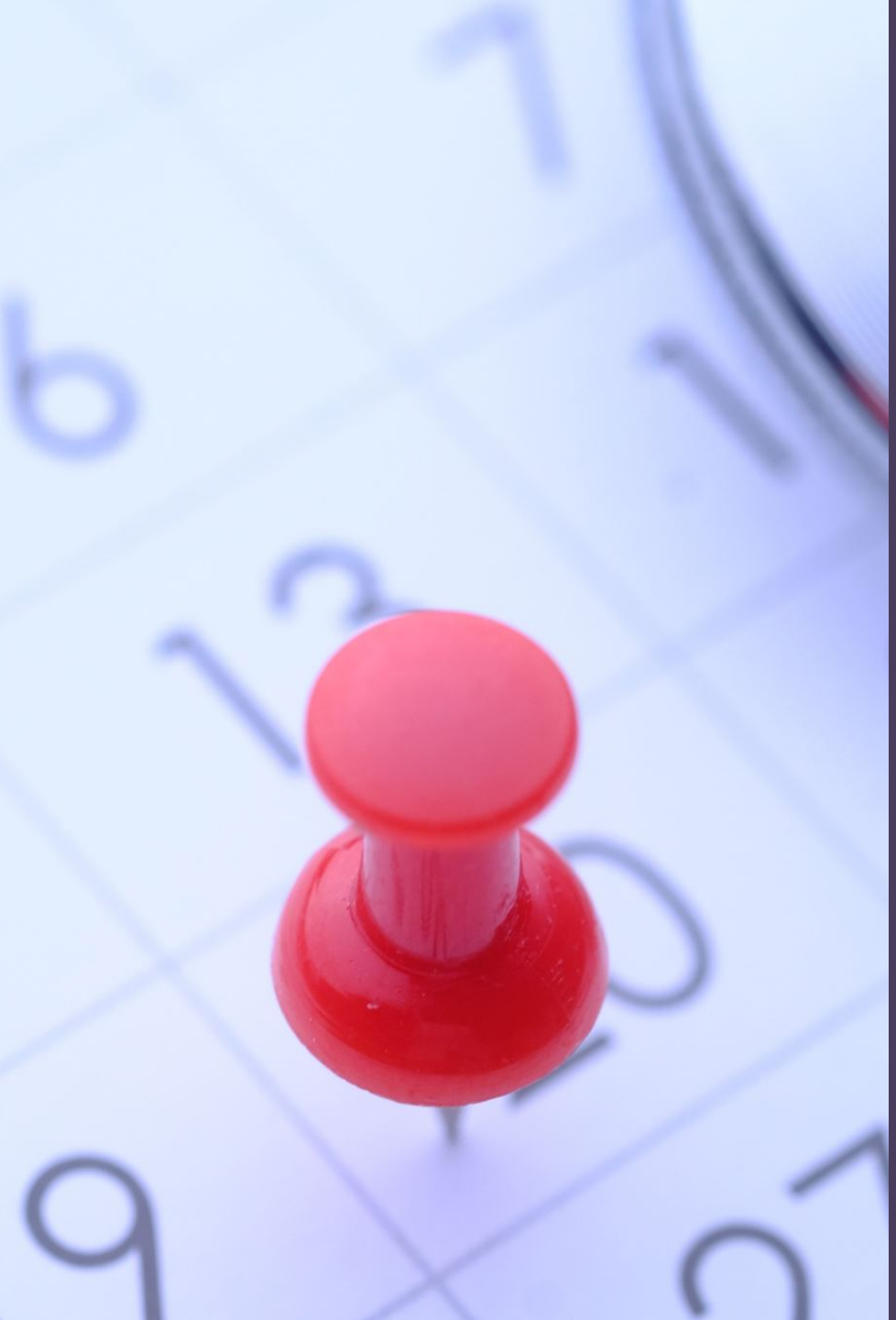
Events

some common HTML events:

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

```
<p id="p1">Hello world</p>
<button onclick="document.getElementById('p1').style.color='red'">
Go RED</button>
```



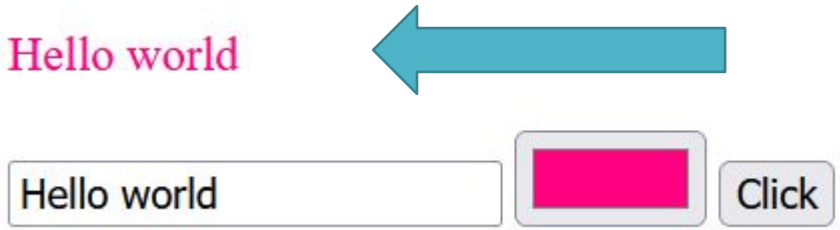


To Do

1. เปลี่ยนสี button และข้อความ
เมื่อกดปุ่ม



2. แสดงข้อความและสี
ตามที่กำหนด



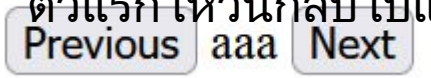
3. แสดงตัวเลข เพิ่ม / ลด
ตามการกดปุ่ม



4. เพิ่มข้อความตามที่ใส่
ใน input



5. แสดงข้อความจาก ข้อ 4. โดย
แสดงข้อความแรกเมื่อมีการเพิ่มข้อความ
จากข้อ 4
Next แสดงข้อความถัดไป หากเกินให้วน
กลับไปแสดงข้อความแรก
previous แสดงข้อความก่อนหน้า หากเป็น
ตัวแรกให้วนกลับไปแสดงข้อความสุดท้าย



6. แสดงรายละเอียดข้อมูล จาก object
ตามที่กำหนด

```
const myObj = {
  name: "John",
  age: 30,
  cars: [
    {name:"Ford", models:["Fiesta", "Focus", "Mustang"]},
    {name:"BMW", models:["320", "X3", "X5"]},
    {name:"Fiat", models:["500", "Panda"]}
  ]
}
```

Owner : John (30)

- Ford
    1. Fiesta
    2. Focus
    3. Mustang
- BMW
    1. 320
    2. X3
    3. X5
- Fiat
    1. 500
    2. Panda

More information

https://www.geeksforgeeks.org/javascript/javascript-tutorial/

https://www.w3schools.com/js/

https://javascript.info/